

OOPS-2

Packages, static, singleton class, Inheritance
Methods

Refer git hub

Packages :-

Packages are containers of classes.
// simple terms are $\rightarrow$ Packages are folders

Use Cases :-

When you have class greeting.java
which prints "hello world". But
you can't create again greeting.java
file in java so by packages
you can create greeting.java in other
package.

Specify the file path in compiler
Package com.kunal. Package.b

Public

Ex :-

```
Package com.kunal.packages.a;
import static com.kunal.packages.b.Message.message;
Public class Greeting {
    Public static void main (String [] args) {
        System.out.println ("hello world");
        message();
    }
}
```

It executes message method from other package

```
Package com.kunal.packages.b;
Public class Message {
```

```
    Public void message () {
```

```
        S.O.P ("This course is awesome");
```

```
    }
}
```

O/P :-

hello world

This course is awesome.

Just because we imported file it runs the method ~~Mass~~ message.

But only class is present as Public is,

import java.util. ~~ArrayList~~
these both are folders at line
java installed

Static Example:-

```
Public class Human {
```

```
    int age;
```

```
    String name;
```

```
    int salary;
```

```
    boolean married; long population;
```

```
    Public human (int age, String name, int salary,  
                  boolean married) {
```

```
        this.age = age;
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
        this.married = married;
```

```
        this.population += 1;
```

```
}
```

What if any properties

```
Public class Main {
```

```
    P.S.V.M {
```

is common for all objects
in classes? $\rightarrow$ static variables

```
        Human kunal = new Human (22, "kunal",  
                                    10000, false);
```

```
        Human rahul = new Human (34, "rahul",  
                                    23000, false);
```

```
        S.O.P (kunal.name);
```

```
        S.O.P (kunal.population);
```

```
        S.O.P (rahul.population);
```

but O/P $\Rightarrow$ 1

answer

1

$\rightarrow$ 2
2

That is why we needed static which is not depend on object

When we changed this to Human (class name)
we get reg. O/P $\rightarrow \frac{2}{2}$

This also works but it better to human (class name) // This is convention

Why this works is because it looks the variable object is present if not it takes from class which is the object variable.

Static variables :-

- * Without reference to any object we can use static variable.

- * You can declare the methods or variables as static.

Public static void main

Why main is declared as static?

Because we can use main method without creating object in the class and also to run the method.

Static method or variable belong to class not to any object.

```
void greeting() {
    cout << "hello world"
}
```

But when we call ~~class~~ greeting() in main class it shows error

error :-

Non static method can be referenced in static method.

// we know something which is not static belongs to an object

```
void greeting () {
```

```
    cout << "hello world";
```

```
}
```

// This is not dependent on objects
static void fun () {

```
    greeting (); // we can't use it requires an instance
```

// but the function you are using it in does not depend on instances

But we can add fun () → method to main

To make it work :- Create an obj

```
static void fun () {
```

```
    Main obj = new Main ();
```

```
    obj.greeting ();
```

// you cannot access non static stuff

without referencing their instances in a static content

Non static members cannot be access without object.

```
void fun2 () { // object needed
```

```
    greeting (); // need object here
```

```
}
```

In main class

```
Main funn = new Main () → obj created
```

```
funn.fun2 ();
```

called

But in

```
static void fun () {
```

greeting (); → Won't run as static

Can't create obj
so they called obj

```
Main obj = new Main ();
```

```
obj.greeting ();
```

At same time

```
static void message () {
```

```
System.out.println("hello world");
```

```
System.out.println(this.age);
```

cannot be used

this cannot be used inside static as
it points to object

Static Init class

// this is done to show initialization of static variables

```
public class StaticBlock {
```

```
static int a = 4;
```

```
static int b;
```

```
// Static Block
```

```
static {
```

Will only run once when the first obj is created in class

```
System.out.println("I am static block");
```

```
b = a * 5;
```

```
} // End of Static Block
```

```
StaticBlock obj = new StaticBlock ();
```

```
System.out.println("Static Block a + " + StaticBlock.a + " + StaticBlock.b");
```

```

StaticBlock.b += 3
    sout (StaticBlock.a + " " + StaticBlock.b);
StaticBlock obj2 = new StaticBlock();
    sout (StaticBlock.a + " " + StaticBlock.b);

```

O/P:-

This is Static
I am static Block

→ It runs only once

4 20

4 203

4 23

Inner classes

In Inner classes outside classes cannot be static

static ~~class~~ A Public class A {

→ not be static error on J

Public class Inner classes?

class Test {

String name;

Public Test (String name) {

this.name = name;

}

}

PSVM {

Test a = new Test ("Kunal");

}

}

→ error as
this cannot be referenced by
static context

Because it depends on Inner class

if we add static to class Test error
will be gone

Q9 put class Test outside the class
inner class and declare String name
as static

```

Public class InnerClass {
    static class Test {
        String name;
        Public Test (String name) {
            this.name = name;
        }
        Public String toString () {
            return name;
        }
    }
}

```

PSVM (String) { }
 Test a = new Test ("Kunal");
 Test b = new Test ("Rahul");
 sout (a.name);
 sout (b.name);
 }
 sout (a);

O/P:-

Kunal
Rahul

We had a doubt static
does not have object then
how different values for
both object because
String is not static
and the inner class not
depend on outer class objects.

System . out . println
 ↓ ↓ ↓
 class variable method

Singleton Class

Only one Instance is allowed
like only one object is allowed

```

Public class Singleton {
    private Singleton () {
    }

    private static Singleton instance;

    public static Singleton getInstance () {
        // check whether 1 obj only is created
        // created or not
        if (instance == null) {
            instance = new Singleton ();
        }
        return instance;
    }
}

```

→ You need constructor to create object so if put private at on that class only the singleton object can be created

Class Main {

PSVM {

```

Singleton obj = Singleton.getInstance (); obj: Singleton
Singleton obj2 = Singleton.getInstance (); obj: Singleton
Singleton obj3 = Singleton.getInstance (); obj: Singleton

```

↓
Reference variables

// all 3 ref variables are pointing to just one object

Mock drill

1. What is Package?

- * A package is a namespace that organizes classes and interfaces. It helps in:
 - i) Avoiding name conflicts
 - * Access Control
 - * Reusability

2. What is difference between

- ans
- i) `import java.util.*;`
 - ii) `import java.util.ArrayList;`
- 5
- i) imports all classes (not sub packages)
 - ii) imports only that class

3. Can we have same class name in different packages?

Yes

```
Package P1;           Package P2;
class A { }           class A;
```

4. Static variables vs Instance variables?

Static variables are independent from the objects. Like if we declare static variables in class it is common for all the objects that use the class. But for instance variables it differs based on object.

Static → Class level (one copy)

Instance → Object level (multiple copies)

5. main method without static?

- * JVM calls `main()` without creating object.
- * If `main` is not static → JVM cannot call it directly.
- * So program won't start.

6. static method & non - static variable :-

- i) static methods belong to class
- ii) Non - static variable belong to objects
- iii) static method cannot directly access non - static variables because no object context exist.

7. static block? - when exactly does it execute?

- i) static block execute once when class is loaded into memory.
- ii) Runs before main()
- iii) Used for initialization of static variables - Execute only once.

8. class Test {

static int x = 10;

static {

x = x + 5;

}

public static void main (String[] args) {

System.out.println(x);

} } ~~ans~~ ans :- 15

9. static method overriding :-

i) static method cannot be overridden

ii) They are method hidden, not overriddable

Ex:- class A {

static void show () {

System.out.println("A");

}

class B extends A {

static void show () {

System.out.println("B");

}

}

A. obj = new B();

obj.show(); // output A

Because static method are resolved at compile time (not run time)

10. Singleton :-

Singleton Class ensures only one object is created for a class and provide a global access point to it. It is implemented by making the constructor private and providing a static method to return the instance.

11. Can we create multiple objects of singleton using reflection?

~~Yes~~ Yes Singleton can be broken using reflection. Because reflection can access private constructor.

Reflection can bypass private constructor and create multiple objects.

12. Difference between static class and singleton class?

Static class :-

- * Cannot create object
- * All members must be static
- * Used for utility method (e.g. Math class)

Singleton class :-

- * Only one object allowed
- * Object is created and reused
- * provides controlled access

Key difference

static → no object at all

singleton → ~~exactly~~ exactly one object

13.

```

class Test {
    static int x = 5;
    static {
        x = x + 2;
    }
    static {
        x = x + 3;
    }
    public static void main (String [] args) {
        System.out.println(x);
    }
}

```

O/P:-

 $x = 5$ $x = 5 + 2 = 10$ // first static block $x = 10 + 3 = 13$ // second static block

14.

System Singleton as bad design:-

- Break OOP principle (tight coupling).
- Hard to test (especially in unit testing)
- Global state $\rightarrow$ can cause bugs in large systems.
- Not flexible (cannot extend easily)
- $\therefore$ Singleton is considered bad because it introduces global state, tight coupling and makes testing difficult.

15.

Logging utility class.

Only method no object needed

static or singleton? why?

ans:- static as it does not need

any objects to execute the code

static is stored in stack memory

as it depend on class.

Since no object state is needed and only behavior is req, static methods are preferred.

Where

static variables are stored?

static $\rightarrow$ method calls, local variables

Heap $\rightarrow$ objects

Method Area $\rightarrow$ static variable, class metadata.

16. Why can't static methods use this keyword?

ans: i) this can be used in any non-static method and constructor.

ii) It refers to the current object

Where this can be used :-

Class A {

int x;

A(int x) {

this.x = x; // constructor

}

void show() {

cout << this.x; // method

}

works in constructor and instance methods.

But not works in

Class A {

static void test() {

cout << this;

}

// Error

ans:- static method belong to the class

and not to any object. Since this

refers to the current object, it

cannot be used in a static context

where no object exists.

17.

```
class Test {
    static int x = 10;
    public static void main (String [] args) {
        System.out.println (Test.x);
        Test t = new Test ();
        System.out.println (t.x);
    }
}
```

3 3
O/P → 10 10

18. Can Constructor be static?
No, constructor is used to initialize objects, so it must be associated with an object and not static.

19. Only one object. But object should be created only when needed. Which Singleton type is lazy initialization Singleton class A?

```
private static A obj;
private A() {}
public static A getInstance () {
    if (obj == null) {
        obj = new A();
    }
    return obj;
}
```

Object created only when needed.

20. Why public static void main?
 public → JVM must access it from anywhere
 static → JVM calls it without creating object
 void → no return value needed
 String [] args → command line arguments

21. If two classes have same name in different packages.

same class

P1. A obj1 = new P1. A ();

P2. A obj2 = new P2. A ();

22. Static Block order

class Test {

O/p

static int x = 5;

static {

Block 1

Block 2

System.out.println("Block1");

Main

} static {

Since static

System.out.println("Block2");

Block order

public static void main (String args[]) {

~~first~~ execute

System.out.println("Main");

when class loaded

}

They run before Main().

23. Can we break Singleton using serialization ?

Yes. Deserialization creates new object use readResolve () method.

24. Integer a = 100;

Integer b = 100;

System.out.println(a == b);

True. True

ans. Both reference to same object

25. class Test {

static int x = 10;

Test () {

O/p

x ++;

12

}

public static void main (String args[]) {

new Test ();

new Test ();

System.out.println(x);

3

?